

Ensembled Models for Text Normalization

A THESIS

SUBMITTED FOR THE DEGREE OF

Master of Technology

IN

Artificial Intelligence

by

Saksham Thakur



Department of Electrical Engineering

Indian Institute of Science

BANGALORE – 560 012

MAY 2026

© Saksham Thakur

MAY 2026

All rights reserved

Abstract

Text normalization (TN) — the conversion of written forms such as numerals, dates, abbreviations, and alphanumeric identifiers into their spoken equivalents — is a fundamental preprocessing step for text-to-speech systems. The task is particularly challenging for morphologically rich Indic languages due to diverse semiotic class structures and the scarcity of publicly available parallel datasets. Existing approaches fall into two categories: handcrafted Weighted Finite-State Transducer (WFST) grammars, which require extensive linguistic expertise, and machine learning models, which demand large-scale parallel data. Critically, rule-based systems excel at capturing generic, pattern-driven transformations, whereas neural models leverage broader contextual cues — a complementary strengths that remains under-exploited in prior work.

In this thesis, we propose a hybrid framework that integrates linguistic knowledge with statistical learning by explicitly combining rule-based and neural normalization strategies. We introduce two ensemble architectures: a sentence-level dynamic model selector that chooses the most appropriate full-sentence normalization from multiple candidates, and a span-level model selector that independently normalizes each semiotic span using the best-performing model for that specific context. On Hindi text normalization, our span-level ensemble achieves state-of-the-art performance, outperforming even supervised fine-tuned large language models such as Gemma (1B) and LLaMA (1B).

Contents

Abstract	ii
1 Introduction	1
2 Dataset Generation	4
3 Overview of Models	6
3.1 Rule-Based Approach	6
3.2 Neural Sequence-to-Sequence Normalization	7
3.3 NER with hints based Normlization	8
3.4 Ensembled Approaches	10
3.4.1 Dynamic Model Selector	10
3.4.2 Evaluation Criterion for Model Selection	10
3.4.3 WER Failure Cases Caused by Noisy LLM Ground Truth . . .	11
3.4.4 Best Model Selection Criterion	12
3.4.5 Training Dataset Construction	13
3.4.6 Dynamic Model Router Architecture	13
3.4.7 Training Objective: Multi-Label Soft Target Loss	14
3.5 Span Level Dynamic Model Selector	15
3.5.1 Span-level Model Router Architecture	16
3.5.2 Inference of Span level Dynamic Model Selector	19
3.6 Large Language Models (LLM)	22
4 Results and Conclusions	23
4.1 Conclusion	26
References	27

List of Tables

4.1	Comparison of text normalization models for Hindi.	23
4.2	Comparison of text normalization models for Kannada.	24
4.3	Model Selection Distribution at Sentence and Span Levels For kannada	25
4.4	Model Selection Distribution at Sentence and Span Levels For Hindi .	25
4.5	Tag Distribution Comparison between Kannada and Hindi	25

List of Figures

1.1	Text normalisation example (from [2])	2
2.1	Illustration of synthetic dataset generation for text normalization . .	4
2.2	Illustration of synthetic dataset generation for text normalization . .	5
3.1	Example of "Span aware hint normalizer" model working	9
3.2	Illustration of Ensemble Model 1 incorporating a dynamic model selector, which evaluates multiple candidate outputs and selects the most appropriate normalized sentence for the given input.	11
3.3	Examples where WER fails due to minor character-level inconsistencies	12
3.4	Example how ensembled model using span level dynamic model selector creates normalized output	15

Chapter 1

Introduction

The work presented here focuses on converting entities in a language that are written in a form different from how they are spoken into their corresponding spoken representations as shown in figure 1 and is a key preprocessing task [10, 19, 2, 12] for Text to Speech (TTS). Text normalization for Hindi remains relatively under-explored compared to English. Text normalization has been studied using rule-based [3, 11] these based methods rely on hand-crafted grammars and transformation rules that explicitly describe how different textual patterns should be converted to spoken written. These systems produce deterministic outputs, but they require significant linguistic expertise to create and are limited to rules set in the production, statistical [6] treat normalization as a probabilistic mapping problem, learning the most likely spoken form from annotated examples, these model quality depends heavily upon the quality of labeled data and especially the domain for which task is aimed at.

The neural approach, as presented in [13, 10], offers greater generalization ability than previous methods but may produce more unpredictable outputs. Still rule based model remain competitive as demonstrated in Zilio et al. (2024) [21], when rule-based models were directly compared against neural approaches, the rule-based system performed competitively with many domain-adapted neural models. It even

outperformed some of them, highlighting the enduring value of rule-based methods for structured normalization tasks. Hybrid approaches combine neural and rule based approaches across different languages and domains. A comprehensive survey of existing techniques (Aliero et al., 2023)[1]. By applying text normalization, a Text To Speech (TTS) system receives only normalized word sequence as input and does not need to learn the mapping from unnormalized entities to their spoken forms, thereby reducing learning burden. Sentences consist of plain and special words or phrases, also known as semiotic tokens [14]. Semiotic tokens can be grouped into classes such as cardinals, date and would require some transformation to be converted to spoken form while non semiotic words are left as it is. We categorize such unnormalized entities into the following subcategories: **Date**, **Money**, **Measure**, **Fraction**, **Decimal**, **Cardinal**, **Telephone**, and **Time**.

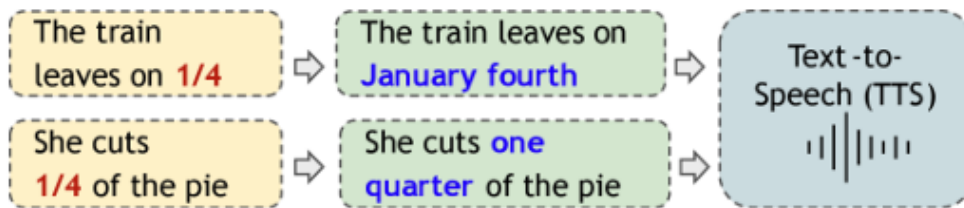


Figure 1.1: Text normalisation example (from [2])

We first propose a **span aware hint normalizer** model that integrates both rule-based cues and neural context, allowing the system to leverage deterministic linguistic rules while still capturing broader contextual patterns in the input text. While this approach combines the strengths of rule-driven precision and neural contextual awareness, its still a merge of both worlds and for every case hints-based model is not the best performing model. In strictly pattern-governed cases [4] provide a strong evidence that rule based model may perform better , whereas in highly contextual scenarios, a pure sequence-to-sequence neural model may offer better generalization. This contextual advantage of neural approaches is explicitly

acknowledged in Bakhturina et al. (2022) [2], who note that while WFST-based rule systems 'struggle with ambiguous input when the normalized form is context-dependent,' neural text normalization systems 'can take context into account.' Their proposed hybrid architecture—which uses a neural language model specifically to 'resolve contextual ambiguity'—further validates that neural models are better suited for grasping the nuanced contextual patterns that rule-based systems cannot easily codify. To address these remaining gaps, we introduce two **ensemble frameworks** that jointly considers the outputs of the hint-based model and rule based and pure neural normalization strategies and provides a suitable normalized output. These ensemble model aims to mitigate the errors arising from either rule or neural components alone and yields improved performance over the standalone hint-based model.

To support these modules, we present a synthetic datageneration pipeline.

Chapter 2

Dataset Generation

Due to the lack of publicly available data, the dataset was derived from the Kestrel [5] dataset, which is a text normalization dataset in the English language. In English, all categories for each individual sample are already available along with the corresponding ground truth. The unnormalized and normalized texts were first translated using Google’s No Language Left Behind (NLLB) model [17]. It was observed that the translation process left the unnormalized parts unchanged. Subsequently, using chain-of-thought prompting, the unnormalized spans along with their categories were identified using a large language model (Gemini [15]). This tagged data was then again provided to the LLM, along with few-shot examples, to generate the corresponding unnormalized and normalized dataset in the target language. This resulted in a fast and cost-effective way to obtain text normalization

Original Sentence (From Kestrel Dataset)	The value increased to 3.75 today at 4:30
Translated Sentence (Using Meta NLLB)	आज मूल्य 3.75 तक बढ़ गया और 4:30 बजे हुआ।
Tagged Sentence (generated using Gemini with example-based chain-of-thought prompting)	आज मूल्य <3.75><DECIMAL> तक बढ़ गया और <4:30><TIME> बजे हुआ।
Normalized Sentence (generated using Gemini with example-based chain-of-thought prompting)	आज मूल्य <तीन दशमलव सात पाँच><DECIMAL> तक बढ़ गया और <चार बजकर तीस मिनट><TIME> बजे हुआ।

Figure 2.1: Illustration of synthetic dataset generation for text normalization

data for Hindi and Kannada and can be extended to other Indian languages as well. For Hindi, approximately 180,000 sentences were generated through the proposed pipeline, with an additional 50,000 sentences obtained directly from the LLM via category-specific prompting. For Kannada, the pipeline yielded around 150,000 sentences, while the remaining 80,000 via category-specific prompting. Notably, the LLM-generated sentences exhibited lower sentence-level variation but higher variation in their unnormalized components.

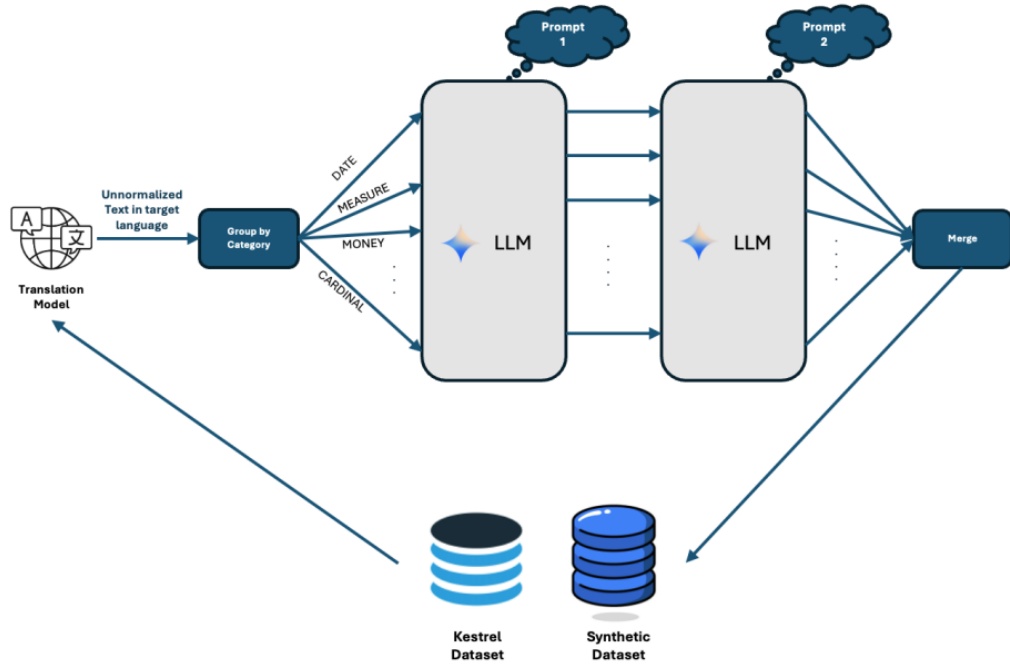


Figure 2.2: Illustration of synthetic dataset generation for text normalization

Chapter 3

Overview of Models

In this chapter, we describe the different approaches explored to address the problem of text normalization. We begin with a traditional rule-based system as a deterministic baseline. We then formulate text normalization as a sequence-to-sequence learning problem, treating it as a machine translation task where the input is an unnormalized sentence and the output is its normalized spoken-form representation. Next, we consider a pipeline-based approach that first identifies unnormalized spans within the input text and subsequently normalizes only those spans. Then we move on to make an ensemble of models comprising of all past models. Finally, we compare these classical and neural approaches with fine-tuned large language models (LLMs) trained on our dataset.

3.1 Rule-Based Approach

The first approach is a fully rule-based text normalization system implemented entirely in Python. This approach does not rely on machine learning; instead, it employs handcrafted linguistic rules and pattern-matching heuristics to identify and normalize non-standard words.

The system supports ten semiotic categories, namely: *dates*, *time*, *measures*,

fractions, cardinals, ordinals, telephone numbers, currency, alphanumeric tokens, and generic numbers. For each category, a deterministic set of transformation rules is defined to convert the written form (e.g., “31-08-1995”, “12km”, “3½”) into its fully expanded spoken representation in the target language.

A major challenge in rule-based text normalization is category ambiguity, as a single token may correspond to multiple interpretations depending on context. For example, “10-05” may represent a date (10 May), a numerical range, or a score. To mitigate this issue, the system incorporates a context-aware category resolution mechanism. When a token matches multiple category patterns, the model examines its surrounding lexical context to infer the most likely category. For instance, words such as “on”, “born”, “dated”, or month names strongly indicate the *DATE* category, whereas cues such as “km”, “kg”, “percent”, or measurement-related verbs bias the classification toward the *MEASURE* category.

Although inherently limited by its dependence on predefined patterns and rules, the system provides a reliable and interpretable baseline for handling well-structured cases across all supported semiotic categories.

3.2 Neural Sequence-to-Sequence Normalization

Here, we handle the normalization task as a machine translation problem. Here, the model completely learns from the data and doesn’t exploit patterns in language that can be used for normalization task. Google’s mT5 [8] (Multilingual Text-to-Text Transfer Transformer) is a sequence-to-sequence language model trained on the large-scale multilingual mC4 corpus, covering more than 100 language. Due to its large-scale pretraining it was selected as one of the models to be used further in our ensembled models.

3.3 NER with hints based Normalization

Here task is divided into two tasks, first identifying the span of token to be normalized then normalizing that part.

Span detection

Given a sentence $S = \{t_1, t_2, \dots, t_n\}$, where t_i denotes the i -th token, the model assigns a corresponding tag to each token.

The tag is 'O' for token which is already in normalized form. For the remaining categories two type of tags are used. For example 'B-Dates' would indicate starting of the 'Date' part and 'I-Dates' would be tag for remaining part this is done for all categories which were listed before which are to be normalized. Table 1 shows examples of such sentences in english language.

Indic-Bert [9] is trained for above name entity recognition(NER) task. tegories. This is then passed with hints to decoder.

Span detection example

Sentence: I paid 1500\$ on 15 January 2023.

MONEY: 1500 (B-MONEY) & \$ (I-MONEY)

DATE: 15 (B-DATE) & January (I-DATE) & 2023 (I-DATE)

Decoder

We convert IndicBERT, originally a bidirectional encoder , into a conditional autoregressive text generator for normalization

1. **Encoder** $\rightarrow$ **Decoder:** The bidirectional attention mechanism is replaced with causal attention so the model predicts tokens sequentially (left-to-right), behaving like a GPT-style model.

2. **Objective Change:** The masked language modeling objective is replaced with next-token prediction, enabling full sequence generation instead of masked token recovery.
3. **Prompt Conditioning:** The input is structured as:

Language + Entity Type + Hint + Original Span

Thus the model learns

$$P(\text{normalized text} \mid \text{language, tag, hint, span}).$$

4. **Training:** The model is trained using causal language modeling loss on the normalized output.

Summary: IndicBERT first performs Named Entity Recognition (NER) on un-normalized text to identify spans requiring normalization and their semiotic types. These spans, along with hints from a rule-based system, are passed to a modified decoder derived from IndicBERT, which generates the final normalized output. Figure 3 illustrates this hybrid hint-based normalization system.

Original Sentence	१५.५.२०२३ को शाम ५:३० बजे को ४.५% ब्याज दर घोषित हुई
NER OUTPUT	Span 1: १५.५.२०२३ → DATE Span 2: ५:३० बजे → TIME Span 3: ४.५% → MEASURE (Percent)
HINTS	DATE: पंद्रह . पाँच . दो हजार तेईस TIME: पाँच:तीस बजे MEASURE: चार . पाँच %
DECODER INPUT PROMPT	[Hindi] [DATE] [पंद्रह . पाँच . दो हजार तेईस] [१५.५.२०२३] [Hindi] [TIME] [पाँच:तीस बजे] [५:३० बजे] [Hindi] [MEASURE] [चार . पाँच %] [४.५%]
DECODER OUTPUT	DATE → पंद्रह मई दो हजार तेईस TIME → पाँच बजकर तीस मिनट MEASURE → चार दशमलव पाँच प्रतिशत
FINAL OUTPUT	पंद्रह मई दो हजार तेईस को शाम पाँच बजकर तीस मिनट को चार दशमलव पाँच प्रतिशत ब्याज दर घोषित हुई

Figure 3.1: Example of "Span aware hint normalizer" model working

The unnormalized part is then replaced with this normalized output.

3.4 Ensembled Approaches

Now we propose two hybrid approaches which combining all past approaches [20]. These approaches combines all previously described normalization systems within a unified framework, first one uses a *Dynamic Model Selector* at sentence level, whereas other uses *Span level Dynamic Model Selector* for each unnormalised span.

3.4.1 Dynamic Model Selector

We employ a *Dynamic Model Selector* (also referred to as a *model router*) to determine the most suitable normalization model for each input sentence. The selector operates over the outputs produced by the following three candidate models:

1. A fully rule-based normalization system,
2. An mT5-based sequence-to-sequence neural normalization model,
3. An IndicBERT-based hints-driven normalization model.

Given an unnormalized input sentence and the corresponding normalized outputs generated by all three models, the selector predicts which candidate is expected to yield the most accurate final normalization. This enables the system to leverage the complementary strengths of symbolic rules, neural sequence modeling, and contextual hint-based normalization in a data-driven manner.

3.4.2 Evaluation Criterion for Model Selection

Selecting an appropriate evaluation metric is critical for both training and validating the dynamic model selector. Although Word Error Rate (WER) is widely used in text normalization and speech processing tasks, we found it to be unreliable in our setting due issues highlighted below.

Unnormalized Sentence: The man ate 567.88 wings in 3.7 hours.

M1 output: The man ate five hundred sixty seven point eight eight wings in three point seven hours.

M2 output: The man ate five hundred and sixty seven point eighty eight wings in three point seven hours

M3 output: The man ate five hundred sixty seven point eight eight wings in three hours and seven minutes.

Model Output Picked by Dynamic Model Selector: M1

Figure 3.2: Illustration of Ensemble Model 1 incorporating a dynamic model selector, which evaluates multiple candidate outputs and selects the most appropriate normalized sentence for the given input.

3.4.3 WER Failure Cases Caused by Noisy LLM Ground Truth

Since the ground truth labels were generated using an LLM, they exhibit minor inconsistencies in spelling, spacing, punctuation, and formatting. WER treats such differences as full word substitutions or insertions, leading to disproportionately high error scores.

Example 1: Nukta Inconsistency In this case, the only difference is the presence or absence of a nukta character. WER counts this as a complete word substitution, whereas CER penalizes only a single character error.

Example 2: Extra or Missing Space A single extra whitespace causes WER to segment the sentence into additional tokens, resulting in multiple word errors. CER, on the other hand, detects only one additional character.

Example 3: Spelling Variation (LLM Ground Truth Noise) The LLM-generated ground truth sometimes contains spelling variations that do not change the semantic meaning of the sentence. Despite producing a linguistically correct

Nukta error:

“हज़ार” vs “हजार”

Noisy ground truths:

“इक्कीस” vs “इकिस”

“इक्कावन” vs “इकावन”

Extra Space in Models outputs:

“पंद्रह सौ” vs “पंद्रह सौ”

Figure 3.3: Examples where WER fails due to minor character-level inconsistencies

output, models—especially the rule-based system—were unfairly penalized by WER due to these inconsistencies.

Summary These examples demonstrate that WER is unstable and misleading when the ground truth contains minor inconsistencies. The Character Error Rate (CER), being a character-level metric, remains robust to such variations and provides a more faithful measure of normalization quality. Therefore, CER is used as the sole metric for selecting the best model and for training the dynamic model selector.

3.4.4 Best Model Selection Criterion

For each input sentence, all three candidate models generate normalized outputs. The Character Error Rate (CER) is computed between each model output and the ground truth.

The model(s) achieving the minimum CER are considered the *best model(s)* for

that sentence. These labels are used as supervision for training the dynamic model selector.

3.4.5 Training Dataset Construction

The base dataset consists of 25,000 Kestrel sentences. To improve generalization and increase numeric diversity, the dataset was expanded using controlled augmentation.

For each original sentence:

- A connector word is randomly sampled from a curated list of 50 commonly used discourse connectors.
- A random numeric pair is generated.
- The connector and numeric pair are appended either to the beginning or the end of the sentence.

The corresponding ground-truth normalized sentence is updated by converting the appended numbers into their fully spoken textual form using the same normalization rules as the original dataset. This ensures consistency while introducing additional syntactic and numeric complexity.

3.4.6 Dynamic Model Router Architecture

The router is implemented as a meta-classifier that predicts the optimal normalization model.

Input Representation The router receives:

- The unnormalized input sentence,
- The normalized outputs from the three candidate models.

Feature Extraction The unnormalized sentence is encoded using an MT5-small Transformer encoder to obtain a contextual embedding and then mean pooled. Each of the three model outputs is encoded using the same encoder architecture.

All four embeddings are concatenated to form a single feature vector:

$$\mathbf{h} = [\mathbf{h}_{\text{input}} \parallel \mathbf{h}_{\text{MT5}} \parallel \mathbf{h}_{\text{IndicBERT}} \parallel \mathbf{h}_{\text{Rule}}]$$

Classification Layer The concatenated feature vector is passed through a linear layer to produce logits:

$$\mathbf{z} = W\mathbf{h} + b$$

where each dimension of $\mathbf{z}$ corresponds to one candidate model.

Output A softmax operation converts logits into probabilities indicating the likelihood of each model being the best choice.

3.4.7 Training Objective: Multi-Label Soft Target Loss

In many cases, multiple models achieve the same minimum CER for a sentence. Therefore, the supervision signal is multi-label. For each sentence, all models achieving the minimum CER are marked as valid targets. Let $\mathcal{B}$ be the set of indices corresponding to the best models. The training loss is defined as:

$$\mathcal{L} = -\frac{1}{|D|} \sum_{i \in D} \left(\frac{1}{|B_i|} \sum_{c \in B_i} \log(p_{i,c}) \right) \quad (3.1)$$

- D represents the dataset, and B_i denotes the set of correct model choices for sample i .
- $p_{i,c}$ is the predicted probability assigned to class c for sample i , obtained using a softmax over the logits.

- $|B_i|$ is the number of valid classes for sample i .

This formulation penalizes the router only when it fails to assign high probability to any of the equally optimal models, allowing it to learn robust decision boundaries between overlapping model strengths. The trained dynamic model selector enables sentence-level adaptive routing, ensuring that the normalization system consistently leverages the most suitable model while remaining robust to noisy ground truth annotations

3.5 Span Level Dynamic Model Selector

Since each unnormalized sentence may contain multiple unnormalized spans, rather than selecting a single model output for the entire sentence, it is more effective to choose the most appropriate normalized span from each model individually. These selected normalized spans can then be used to replace the corresponding unnormalized spans in the original sentence, thereby forming a fully normalized sentence. The best model for each span should be determined based on the lowest Character Error Rate (CER) when comparing its normalized output to the ground truth of that specific span.

Unnormalized sentence	The event is on <12/03/2024><span1> and costs <\$1200><span_2>.
Model 1	The event is on twelfth March two thousand twenty four and costs one thousand two hundred .
Model 2	The event is on December third two thousand twenty four and costs twelve hundred dollars
Model 3	The event is on one two zero three two zero two four and costs one thousand two hundred
Merged output	The event is on < twelfth March two thousand twenty four ><span_1 from M1> and costs < twelve hundred dollars ><span_2 from M2>

Figure 3.4: Example how ensembled model using span level dynamic model selector creates normalized output

3.5.1 Span-level Model Router Architecture

The MT5SpanSelector is a neural architecture that selects the best normalization model for each text span in Hindi text normalization. The router operates at the span level, dynamically choosing among three candidate models (MT5, HINTS-IB, and RB) based on the context and the quality of each model’s output.

Architecture Overview The model employs a pretrained MT5 encoder as the backbone, processing five distinct inputs per span through separate projection layers before combining them for classification.

Input Representation For each span that requires normalization, the model accepts five input components:

1. **Sentence Context:** The complete raw sentence containing the span
2. **Raw Span:** The unnormalized text segment (e.g., “539”)
3. **RB Output:** RB model’s normalized version of the span
4. **MT5 Output:** MT5 model’s normalized version of the span
5. **HINTS-IB Output:** HINTS-IB model’s normalized version of the span

Each input is independently encoded using the shared MT5EncoderModel.

Encoding Mechanism Given an input text x (sentence or span), the model encodes it using mean pooling over the MT5 encoder outputs:

$$\mathbf{h}_x = \frac{1}{|T|} \sum_{t \in T} \text{MT5Encoder}(t) \quad (3.2)$$

where T is the set of tokens after tokenization, and $\text{MT5Encoder}(t) \in \mathbb{R}^{512}$ is the token embedding. This produces a fixed-size representation of 512 dimensions for any input length.

Projection Layers Each encoded component passes through its own linear projection layer:

$$\mathbf{p}_{\text{sent}} = W_{\text{sent}} \mathbf{h}_{\text{sent}} + b_{\text{sent}} \quad (3.3)$$

$$\mathbf{p}_{\text{raw}} = W_{\text{raw}} \mathbf{h}_{\text{raw}} + b_{\text{raw}} \quad (3.4)$$

$$\mathbf{p}_{\text{rb}} = W_{\text{rb}} \mathbf{h}_{\text{rb}} + b_{\text{rb}} \quad (3.5)$$

$$\mathbf{p}_{\text{mt5}} = W_{\text{mt5}} \mathbf{h}_{\text{mt5}} + b_{\text{mt5}} \quad (3.6)$$

$$\mathbf{p}_{\text{hints}} = W_{\text{hints}} \mathbf{h}_{\text{hints}} + b_{\text{hints}} \quad (3.7)$$

where $W. \in \mathbb{R}^{512 \times 512}$ and $b. \in \mathbb{R}^{512}$. These projections allow the model to learn task-specific transformations for each input source.

Feature Concatenation All five projected representations are mean pooled and then these are concatenated along the feature dimension:

$$\mathbf{c} = [\mathbf{p}_{\text{sent}}; \mathbf{p}_{\text{raw}}; \mathbf{p}_{\text{rb}}; \mathbf{p}_{\text{mt5}}; \mathbf{p}_{\text{hints}}] \in \mathbb{R}^{2560} \quad (3.8)$$

This concatenation preserves all information while allowing the model to learn interactions between different input sources.

Combine MLP The concatenated features pass through a two-layer MLP:

$$\mathbf{z}_1 = \text{ReLU}(W_1 \mathbf{c} + b_1), \quad W_1 \in \mathbb{R}^{1024 \times 2560} \quad (3.9)$$

$$\mathbf{z}_1 = \text{Dropout}(\mathbf{z}_1) \quad (3.10)$$

$$\mathbf{z}_2 = \text{ReLU}(W_2 \mathbf{z}_1 + b_2), \quad W_2 \in \mathbb{R}^{512 \times 1024} \quad (3.11)$$

$$\mathbf{z} = \text{Dropout}(\mathbf{z}_2) \quad (3.12)$$

This compresses the 2560-dimensional representation into a 512-dimensional feature vector that captures high-level interactions between the span context and each candidate model’s output.

Classification Head The final classifier produces logits for the three candidate models:

$$\mathbf{l} = W_{\text{cls}}\mathbf{z} + b_{\text{cls}}, \quad W_{\text{cls}} \in \mathbb{R}^{3 \times 512} \quad (3.13)$$

The probability distribution over models is obtained via softmax:

$$P(\text{model} = m \mid \text{span}) = \frac{\exp(l_m)}{\sum_{m' \in \mathcal{M}} \exp(l_{m'})} \quad (3.14)$$

where $\mathcal{M} = \{\text{RB}, \text{MT5}, \text{HINTS-IB}\}$.

Training The model is optimized using cross-entropy loss with a modified target distribution to handle ties. For each span, the ground truth label is determined by the Character Error Rate (CER) of each candidate model’s output.

Let $\text{CER}_{n,m}$ denote the CER for span n produced by model $m \in \mathcal{M} = \{\text{RB}, \text{MT5}, \text{HINTS-IB}\}$. The set of models that achieve the minimum CER for span n is defined as:

$$\mathcal{M}_n^* = \{m \mid \text{CER}_{n,m} = \min_{m' \in \mathcal{M}} \text{CER}_{n,m'}\} \quad (3.15)$$

If a single model achieves the lowest CER, that model is the sole correct class. If multiple models achieve the same minimum CER (i.e., a tie), all tied models are considered equally correct.

The target probability distribution for span n is constructed as:

$$t_{n,m} = \begin{cases} \frac{1}{|\mathcal{M}_n^*|} & \text{if } m \in \mathcal{M}_n^* \\ 0 & \text{otherwise} \end{cases} \quad (3.16)$$

The cross-entropy loss for a batch of N spans is then computed as:

$$\mathcal{L} = -\frac{1}{N} \sum_{n=1}^N \sum_{m \in \mathcal{M}} t_{n,m} \log P(\text{model} = m \mid \text{span}_n) \quad (3.17)$$

where $P(\text{model} = m \mid \text{span}_n)$ is the probability predicted by the model.

This formulation has two key properties:

- When a single model has the lowest CER, the loss reduces to standard cross-entropy against that model as the unique correct class.
- When multiple models tie for the lowest CER, the loss encourages the model to distribute its probability equally among all tied models, rather than forcing it to choose one arbitrarily.

This tie-aware cross-entropy loss prevents the model from being penalized when multiple models produce equally valid normalizations, which is common in text normalization where different surface forms may be equally acceptable

3.5.2 Inference of Span level Dynamic Model Selector

The inference pipeline identifies spans in the raw text that require normalization by comparing the raw input directly with each candidate model’s output and aggregating the differences through a voting mechanism.

Tokenization and Alignment For each input sentence, both the raw text and each candidate model output are tokenized based on whitespace, while preserving the original character positions of each token. No additional normalization or pre-processing is applied prior to alignment, ensuring that the original structure of the text is maintained.

A token-level alignment is then performed between the raw text and each model output using a sequence matching algorithm. The alignment identifies segments where the two sequences differ, which correspond to potential normalization regions.

Each alignment operation is categorized implicitly as either:

- **Equal segments:** Tokens that match exactly between raw text and model output
- **Non-equal segments:** Tokens that differ, primarily corresponding to replacements or deletions in the raw text. Pure insertions in the model output (with no corresponding source tokens) are not treated as standalone spans.

For each non-equal segment, the system extracts:

- The span in the raw text using character-level start and end indices
- The corresponding normalized text from the model output

Span Aggregation and Voting After processing all candidate models, the system aggregates spans across models using their character ranges. Each unique span is identified by its start and end positions in the raw text.

For each span, the system:

1. Collects normalized outputs proposed by different models
2. Counts the number of models that identified the span (vote count)
3. Determines whether all models both identify the span and agree on the normalized output (consensus)

A higher vote count indicates stronger agreement among models that the span requires normalization. Spans proposed by a single model are also retained and considered valid candidates.

Overlap Resolution Since spans from different models may overlap, the system applies a greedy left-to-right conflict resolution strategy. Overlapping spans are resolved by:

- Preferring spans with higher vote counts
- In case of ties, selecting the longer span

This results in a non-overlapping set of spans for reconstruction.

Span Representation Each finalized span includes:

- The unnormalized span text
- Start and end character positions
- A mapping of model names to their proposed normalized outputs
- The vote count
- A consensus flag indicating agreement across models

Integration with Classification The identified spans are passed to a classification model. For each span, the classifier receives:

- The raw span text
- Candidate normalized outputs from each model
- Sentence-level contextual information provided implicitly through the original input sentence

The classifier predicts which model output is most appropriate for each span.

Reconstruction The final normalized sentence is constructed by replacing each identified span in the original text with the normalized output selected by the classifier. The reconstruction preserves the original text structure outside the identified spans.

Edge Case Handling The system handles several edge cases:

- If no spans are detected, the original input is returned unchanged
- Spans identified by a single model are still considered valid
- Models that do not propose a replacement for a span are ignored for that span

3.6 Large Language Models (LLM)

With the availability of smaller open-source large language models (LLMs), these models also present a viable option for text normalization. However, we observed that their non-fine-tuned versions perform poorly on this task; therefore, we report results only for the fine-tuned variants. In particular, we fine-tuned Gemma-3-1B [16] and LLaMA-3.2-1B [18] using LoRA [7] to evaluate the effectiveness of compact open-source LLMs for text normalization. Larger LLMs were not considered, as text normalization is a preprocessing task where computational efficiency is an important constraint.

Chapter 4

Results and Conclusions

The Hindi test set consists of approximately 10,000 sentences and for Kannada around 8000 sentences. The results are presented in Table 4.1. Here, **Oracle 1** represents the upper bound of the sentence-level ensemble approach, where an ideal dynamic model selector perfectly chooses the best-performing model output for each sentence. In contrast, **Oracle 2** represents the upper bound of the span-level ensemble approach, assuming perfect span detection and optimal selection of the best-performing model for each span.

Table 4.1: Comparison of text normalization models for Hindi.

Model	CER (%)	WER (%)
Rule-based (RB)	8.08	14.71
mT5	7.65	12.41
mT5 Large	6.20	10.88
IndicBART	6.55	10.68
Span Aware Hints Normalizer/HINTS-IB	6.32	10.34
LLaMA 1B	5.93	10.29
Gemma 1B	7.66	12.76
Ensembled with Span Selector	5.56	10.36
Ensembled with Dynamic Model Selector	5.56	11.12
Oracle 1 (For Ensembled model 1)	3.31	8.50
Oracle 2 (Span-level)	0.56	0.79

From Table 4.1, it can be observed that the proposed ensemble models achieve

Table 4.2: Comparison of text normalization models for Kannada.

Model	CER (%)	WER (%)
Rule-based (RB)	14.35	25.27
mT5	7.23	13.64
mT5 Large	4.23	8.91
Span Aware Hints Normalizer/HINTS-IB	5.96	14.02
LLaMA 1B	4.28	9.45
Gemma 1B	5.37	10.62
Ensembled with Span Selector	5.57	12.44
Ensembled with Dynamic Model Selector	6.03	13.42
Oracle 1 (For Ensembled model 1)	2.86	7.94
Oracle 2 (Span-level)	0.55	0.69

the best overall performance among all evaluated approaches. Furthermore, a comparison with Oracle 1 and Oracle 2 highlights the potential of the ensemble methods. In particular, the gap between the span-level ensemble and Oracle 2 indicates significant room for improvement, suggesting that more accurate span identification and selection could further enhance performance.

From Table 4.2, it can be observed that the proposed ensemble methods perform comparatively weaker on Kannada than on Hindi. Although the span-level ensemble (Ensembled with Span Selector) outperforms each of its individual component models, it still falls short of stronger models such as mT5 Large and fine-tuned LLMs. Given that the training and inference strategy remained consistent across languages, this performance gap is likely due to differences in pretraining data availability and quality. This highlights a clear disparity within Indic languages, revealing an imbalance in data readiness and linguistic resource coverage between higher-resource languages like Hindi and lower-resource ones like Kannada.

On the other hand, the sentence-level ensemble (Ensembled with Dynamic Model Selector) provides a modest improvement in overall performance when considering both CER and WER jointly. However, the gains are limited, indicating that the effectiveness of ensemble strategies is more constrained for Kannada, possibly due

to differences in data characteristics or model generalization across languages.

This discrepancy in performance across the two languages is likely attributable to differences in the availability and quality of pretraining data, with Hindi benefiting from richer representation in large-scale corpora compared to Kannada. Additionally, the rule-based model is weaker for Kannada than for Hindi, further contributing to the reduced effectiveness of the ensemble approaches in the Kannada setting.

Table 4.3: Model Selection Distribution at Sentence and Span Levels For kannada

Model	Sentence-level (%)	Span-level (%)
MT5	45.80	67.41
Hints-IB	48.10	13.90
Rule-based	6.10	18.70

Table 4.4: Model Selection Distribution at Sentence and Span Levels For Hindi

Model	Sentence-level (%)	Span-level (%)
MT5	37.52	47.84
Hints-IB	36.34	20.84
Rule-based	26.14	31.32

Table 4.5: Tag Distribution Comparison between Kannada and Hindi

Tag	Kannada (%)	Hindi (%)
CARDINAL	30.65	36.54
DECIMAL	36.22	18.26
DATE	12.21	20.63
MEASURE	9.18	7.19
MONEY	6.93	8.78
TIME	1.16	2.10
TELEPHONE	2.72	2.88
FRACTION	0.73	2.86

4.1 Conclusion

The performance analysis of Oracle 1 and Oracle 2 reveals that there remains considerable room for improvement, particularly in Ensemble Approach 2(Span level Selection). In this approach, a normalized output is constructed by selecting different spans from different models. However, the outputs of the individual models are not substantially different—they are largely similar. Furthermore, the application of mean pooling to construct feature vectors discards fine-grained information, yielding training signals that are notably weak. Despite this limitation, the combination of rule-based methods with machine learning shows clear promise, as evidenced by the performance achieved on Hindi.

References

- [1] Abubakar Aliero, Sulaimon Bashir, Hamzat Aliyu, Amina Tafida, Bashar Kangiwa, and Nasiru Dankolo. Systematic review on text normalization techniques and its approach to non-standard words. *International Journal of Computer Applications*, 185:975–8887, 09 2023.
- [2] Evelina Bakhturina, Yang Zhang, and Boris Ginsburg. Shallow fusion of weighted finite-state transducer and language model for text normalization, 2022.
- [3] Richard Beaufort, Sophie Roekhaut, Louise-Amélie Cougnon, and Cédric Faron. A hybrid rule/model-based finite-state framework for normalizing sms messages. 09 2010.
- [4] Kacper Dudzic, Filip Graliński, Krzysztof Jassem, Marek Kubis, and Piotr Wierzchoń. Two approaches to diachronic normalization of polish texts, 2024.
- [5] Peter Ebdon and Richard Sproat. The kestrel tts text normalization system. *Natural Language Engineering*, 21(3):333–353, 2015.
- [6] Bo Han and Timothy Baldwin. Lexical normalisation of short text messages: Makn sens a #twitter. In Dekang Lin, Yuji Matsumoto, and Rada Mihalcea, editors, *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 368–378, Portland, Oregon, USA, June 2011. Association for Computational Linguistics.

- [7] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models, 2021.
- [8] A. Jha, H. Y. Patil, S. K. Jindal, and S. M. N. Islam. Multilingual indian language neural machine translation system using mt5 transformer. In *Proceedings of the 2nd International Conference on Paradigm Shifts in Communications, Embedded Systems, Machine Learning and Signal Processing (PCEMS)*, pages 1–5, Nagpur, India, 2023.
- [9] Divyanshu Kakwani, Anoop Kunchukuttan, Satish Golla, et al. Indicbert: A multilingual language model for indian languages. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [10] Courtney Mansfield, Ming Sun, Yuzong Liu, Ankur Gandhe, and Björn Hoffmeister. Neural text normalization with subword units. In Anastassia Loukina, Michelle Morales, and Rohit Kumar, editors, *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Industry Papers)*, pages 190–196, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics.
- [11] Zainab Mansur, Nazlia Omar, Sabrina Tiun, and Eissa M Alshari. A normalization model for repeated letters in social media hate speech text based on rules and spelling correction. *PLoS ONE*, 19(3):e0299652, 2024.
- [12] K Panchapagesan, Partha Pratim Talukdar, N Sridhar Krishna, Kalika Bali, and AG Ramakrishnan. Hindi text normalization. In *Fifth International Conference on Knowledge Based Computer Systems (KBCS)*, pages 19–22, 2004.

- [13] Richard Sproat and Navdeep Jaitly. RNN approaches to text normalization: A challenge. *CoRR*, abs/1611.00068, 2016.
- [14] Paul Taylor. *Text-to-Speech Synthesis*. Cambridge University Press, 2009.
- [15] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, and et al. Gemini: A family of highly capable multimodal models, 2025.
- [16] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhu-patiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, Pier Giuseppe Sessa, Aakanksha Chowdhery, Adam Roberts, Aditya Barua, Alex Botev, Alex Castro-Ros, Ambrose Slone, Amélie Héliou, Andrea Tacchetti, Anna Bulanova, Antonia Paterson, Beth Tsai, Bobak Shahriari, Charline Le Lan, Christopher A. Choquette-Choo, Clément Crepy, Daniel Cer, Daphne Ippolito, David Reid, Elena Buchatskaya, Eric Ni, Eric Noland, Geng Yan, George Tucker, George-Christian Muraru, Grigory Rozhdestvenskiy, Henryk Michalewski, Ian Tenney, Ivan Grishchenko, Jacob Austin, James Keeling, Jane Labanowski, Jean-Baptiste Lespiau, Jeff Stanway, Jenny Brennan, Jeremy Chen, Johan Ferret, Justin Chiu, Justin Mao-Jones, Katherine Lee, Kathy Yu, Katie Millican, Lars Lowe Sjoesund, Lisa Lee, Lucas Dixon, Machel Reid, Maciej Mikuła, Matteo Wirth, Michael Sharman, Nikolai Chinaev, Nithum Thain, Olivier Bachem, Oscar Chang, Oscar Wahltinez, Paige Bailey, Paul Michel, Petko Yotov, Rahma Chaabouni, Ramona Comanescu, Reena Jana, Rohan Anil, Ross McIlroy, Ruibo Liu, Ryan Mullins, Samuel L Smith, Sebastian Borgeaud, Sertan Girgin, Sholto Douglas, Shree Pandya, Siamak Shakeri, Soham De, Ted Klimenko, Tom Henni-gan, Vlad Feinberg, Wojciech Stokowiec, Yu hui Chen, Zafarali Ahmed, Zhitao Gong, Tris Warkentin, Ludovic Peran, Minh Giang, Clément Farabet, Oriol Vinyals, Jeff Dean, Koray Kavukcuoglu, Demis Hassabis, Zoubin Ghahramani, Douglas Eck, Joelle Barral, Fernando Pereira, Eli Collins, Armand Joulin, Noah

- Fiedel, Evan Senter, Alek Andreev, and Kathleen Kenealy. Gemma: Open models based on gemini research and technology, 2024.
- [17] NLLB Team, Marta R. Costa-jussà, James Cross, Onur Çelebi, Maha Elbayad, Kenneth Heafield, Kevin Heffernan, Elahe Kalbassi, Janice Lam, Daniel Licht, Jean Maillard, Anna Sun, Skyler Wang, Guillaume Wenzek, Al Youngblood, Bapi Akula, Loic Barrault, Gabriel Mejia Gonzalez, Prangthip Hansanti, John Hoffman, Semarley Jarrett, Kaushik Ram Sadagopan, Dirk Rowe, Shannon Spruit, Chau Tran, Pierre Andrews, Necip Fazil Ayan, Shruti Bhosale, Sergey Edunov, Angela Fan, Cynthia Gao, Vedanuj Goswami, Francisco Guzmán, Philipp Koehn, Alexandre Mourachko, Christophe Ropers, Safiyyah Saleem, Holger Schwenk, and Jeff Wang. No language left behind: Scaling human-centered machine translation, 2022.
- [18] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models, 2023.
- [19] Michel Wong, Ali Alshehri, Sophia Kao, and Haotian He. Polynorm: Few-shot llm-based text normalization for text-to-speech, 2025.
- [20] Junhui Zhang, Junjie Pan, Xiang Yin, Chen Li, Shichao Liu, Yang Zhang, Yuxuan Wang, and Zejun Ma. A hybrid text normalization system using multi-head self-attention for mandarin. In *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 6694–6698, 2020.
- [21] Leonardo Zilio, Rafaela Radünz Lazzari, and Maria José Bocorny Finatto. Can rules still beat neural networks? the case of automatic normalisation for 18th-century portuguese texts. In *International Conference on Computational Processing of the Portuguese Language*, 2024.